Autonomous Robotic Warehouse Inventory & Micro-Fulfillment System

JAVA PROJECT REPORT SUBMITTED IN PARTIAL FULFILLMENT OF THE REQUIREMENTS FOR THE AWARD OF THE DEGREE OF **BACHELOR OF ENGINEERING** IN **ARTIFICIAL INTELLIGENCE AND DATA SCIENCE** OF THE ANNA UNIVERSITY

JAVA PROJECT
NOVEMBER 2026

PROJECT WORK

Submitted by

VARUN.S-722825243235

BATCH
2025 – 2029

Under the Guidance of

Dr.C.Rajeshkumar

Assistant Professor

Department of Computer & Communication Engineering

Sri Eshwar College of Engineering

(An Autonomous Institution – Affiliated to Anna University)

COIMBATORE – 641 202

Sri Eshwar College of Engineering

(An Autonomous Institution – Affiliated to Anna University)
COIMBATORE – 641 202

BONAFIDE CERTIFICATE

Certified that this Report titled **". Autonomous Robotic Warehouse Inventory & Micro-Fulfillment System"** is the bonafide work of

Varun.S	**722825243235**

who carried out the project work under my supervision.

----------	**----------**
SIGNATURE	**SIGNATURE**
Dr.G.Sathish Kumar **HEAD OF THE DEPARTMENT** Artificial Intelligence and Data Science, Sri Eshwar College of Engineering, Coimbatore – 641 202.	Dr.C.Rajeshkumar **SUPERVISOR** Assistant Professor Artificial Intelligence and Data science, Sri Eshwar College of Engineering, Coimbatore – 641 202.

Submitted for the **Autonomous Semester End Mini Project Viva-Voce** held on

……..

_____	**_____**
INTERNAL EXAMINER	**EXTERNAL EXAMINER**

DECLARATION

Varun.S [722825243235]

To declare that the project entitled **". Autonomous Robotic Warehouse Inventory & Micro-Fulfillment System"** submitted in partial fulfilment to the University as the project work of Bachelor of Engineering (Artificial Intelligence and Data Science) Degree, is a record of original work done by us under the supervision and guidance of **Dr.C.Rajeshkumar,** Assistant Professor, Department of Artificial Intelligence and Data Science, Sri Eshwar College of Engineering, Coimbatore.

Place: Coimbatore
Date:

[Varun.S]

Project Guided by,

[Dr.C.Rajeshkumar, AP/AD]

ACKNOWLEDGEMENT

The success of a work depends on a team and cooperation. I take this opportunity to express my gratitude and thanks to everyone who helped me in my project. I would like to thank the management for the constant support provided by them to complete this project.

It is indeed our great honor bounded duty to thank our beloved **Chairman Mr. R. Mohanram,** for his academic interest shown towards the students.

We are indebted to our **Director Mr. R. Rajaram,** for motivating and providing us with all facilities.

I wish to express my sincere regards and deep sense of gratitude to **Dr. Sudha Mohanram, M.E, Ph.D. Principal**, for the excellent facilities and encouragement provided during the course of the study and project.

We are indebted to **Dr.G.Sathish Kumar** Head of Artificial Intelligence and Data Science for having permitted us to carry out this project and giving the complete freedom to utilize the resources of the department.

I express my sincere thanks to my mini project Coordinator **Dr.C.RajeshKumar,** Assistant Professors of Artificial Intelligence and Data Science Engineering Department for the valuable guidance and encouragement given, to us for this project.

I solemnly express our thanks to all the teaching and nonteaching staff of the Artificial Intelligence and Data Science Engineering Department, family and friends for their valuable support which inspired us to work on this project.

TABLE OF CONTENT

CONTENT	PAGE NO
ABSTRACT	1
LIST OF FIGURES	2
LIST OF TABLE	3
1. INTRODUCTION	4
1.1. Objective	5
1.2. Existing System	6
2. SYSTEM ANALYSIS	7
2.1. Problem Statement	8
2.2. Software Requirements	9
3. LOW-LEVEL DESIGN (LLD)	10
3.1. Flow Chart	11
	12
4. IMPLEMENTATION AND TESTING	19
4.1. Code	20
4.2. Output	21
5. CONCLUSION AND FUTURE SCOPE	22
5.1. Conclusion	23
	24
6. APPENDICES AND REFERENCES	25
6.1. REFERENCES	26
6.2. APENDIX A	27
6.3. APENDIX B	28

- **Below are the sample contents for text formatting. Update the project content as per the contents given in table of contents**

ABSTRACT

This paper presents the design and implementation of an Autonomous Robotic Warehouse Inventory & Micro-Fulfillment ERP using Spring Boot, a powerful Java framework for building enterprise applications. The system tracks autonomous mobile robots (AMRs), manages automated bin storage and pick cycles, and handles billing for e-commerce merchants availing fulfillment services. It facilitates CRUD (Create, Read, Update, Delete) operations for AMR fleet management, inventory tracking, and financial ledger entries, allowing fulfillment operators to seamlessly dispatch robots, monitor battery health, and settle merchant accounts without manual intervention. A key feature is its ability to generate consolidated Profit & Loss and Budget Reports comparing fulfillment revenues against robotics maintenance costs, leveraging Spring Data JPA and MySQL to ensure data integrity. The user interface uses HTML, CSS, and JavaScript to provide a clean, real-time dashboard for managing AMR operations and warehouse accounts.

1.1 OBJECTIVE

The Spring Boot-based Autonomous Robotic Warehouse ERP aims to develop a robust web application for automated fulfillment center operations. It prioritizes accurate tracking of AMR telemetry, battery health, and dispatch status, while enabling fulfillment accountants to bill e-commerce merchants for pick/pack and storage services. Users can effortlessly manage robot fleets, inventory bins, and vendor purchase orders through seamless CRUD operations, and gain actionable insights from automated ledger postings and budget comparison reports.

Built with HTML, CSS, and JavaScript, the application provides an intuitive interface for dispatching robots and tracking pick-cycle completion in real time. It emphasizes operational visibility and accountability through structured master data (Contacts, Products, Chart of Accounts) and transaction flows (Purchase Order → Vendor Bill → Payment; Sales Order → Customer Invoice → Payment). Data integrity is maintained using Spring Boot's JPA layer with a MySQL database.

Ultimately, this project empowers fulfillment operations directors and accountants with a unified tool for coordinating robotics infrastructure, warehouse inventory, and financial reporting.

2.1 EXISTING SCENARIO

In the current warehousing landscape, most micro-fulfillment centers rely on either fully manual inventory tracking or expensive proprietary WMS (Warehouse Management System) platforms such as Fetch Robotics or 6 River Systems' ERP integrations. These systems offer robot fleet coordination and billing modules, but come with high licensing costs, rigid configurations, and limited customization for smaller fulfillment hubs. Manual spreadsheet-based tracking, on the other hand, cannot scale with real-time AMR telemetry or automatically reconcile pick/pack revenue against maintenance expenses.

For smaller fulfillment operators who need a customizable, self-hosted system, these mainstream solutions are often impractical. This gap is where a Java Spring Boot-based Autonomous Robotic Warehouse ERP becomes relevant — combining AMR fleet tracking with lightweight accounting, giving operators full control over dispatch logic, master data, and financial reporting without vendor lock-in.

2.2 PROBLEM STATEMENT

Fulfillment centers deploying autonomous mobile robots currently lack an integrated system that ties robot operational data — battery health, dispatch status, and pick completion — directly to financial outcomes such as merchant billing, hardware vendor payments, and maintenance budgets. Operations directors need real-time visibility into AMR fleet status, bin storage layouts, and rack utilization, while fulfillment accountants separately need to bill e-commerce merchants accurately, process robotics component purchase orders, and register vendor receipts. In most existing setups, these functions are handled through disconnected tools or manual spreadsheets, which makes it difficult to reconcile robot maintenance expenses against pick/pack service revenue or to track hub-level budget performance over time.

This project addresses that gap by consolidating AMR fleet tracking, master data management, transaction processing, and budget reporting within a single Spring Boot application. Admins can configure AMR dispatch paths and review storage rack layouts, while invoicing users can bill merchants, process vendor bills for spare parts, and post journal entries — all against a shared Chart of Accounts and Analytic Account structure per fulfillment hub. This unified approach eliminates data silos, gives operations directors and accountants a common source of truth, and enables automated Balance Sheet, Profit & Loss, and Budget Reports that reflect actual robotics performance alongside financial health.

PROPOSED SOLUTION

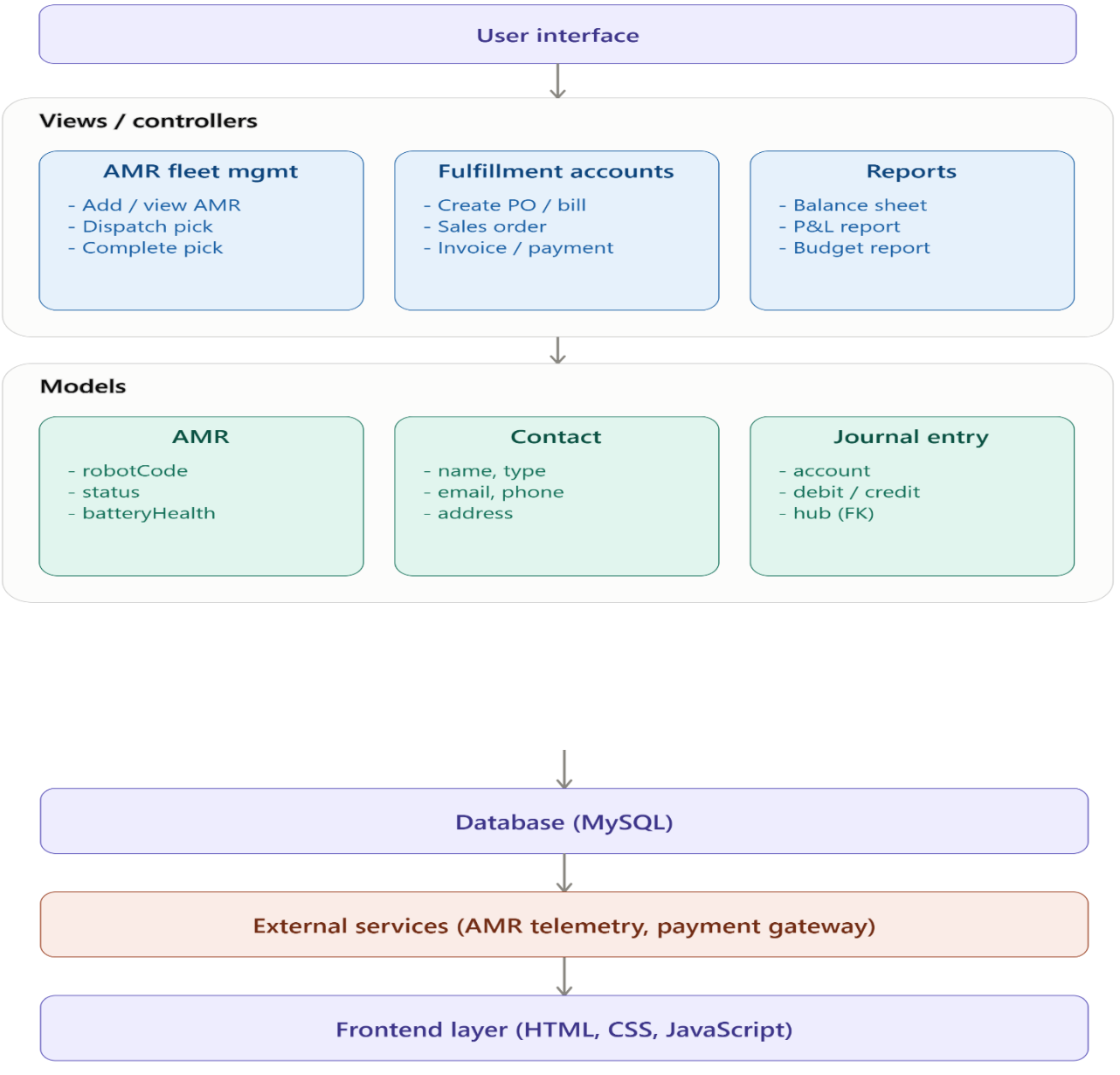
3.1 OVERVIEW

The Spring Boot-based Autonomous Robotic Warehouse ERP unifies fleet operations and fulfillment accounting in one platform. It uses REST controllers to expose AMR management endpoints (/api/fulfillment/amrs) for creating robot records, dispatching pick cycles, and marking picks complete, backed by Spring Data JPA repositories and a MySQL database.

Fulfillment accountants can register e-commerce merchants and AMR hardware vendors under Contact Master, define services (Pick & Pack, Bin Storage) and goods (Battery Cells) under Product Master, and post transactions through a structured Chart of Accounts (Assets, Liabilities, Income, Expenses). The transaction flow supports Purchase Orders converting to Vendor Bills, and Sales Orders converting to Customer Invoices, each settled via Bank/Cash journals.

An Analytic Account per Fulfillment Hub (e.g., Automated Hub Sector 2) enables budget comparison — planned operating budgets versus actual robotics maintenance costs and fulfillment revenues — surfaced through Balance Sheet, Profit & Loss, and Budget Reports. The frontend, built with HTML, CSS, and JavaScript, provides a real-time table view of the AMR fleet with dispatch, complete, and delete actions.

3.2 FLOW CHART



4.1 CODE

CONTROLLER (ExpenseController.java)

```
package com.example.java_project.controller;

import com.example.java_project.entity.amr_entity;
import com.example.java_project.service.amr_service;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;

import java.util.List;
import java.util.Optional;

@RestController
@RequestMapping("/api/fulfillment/amrs")
@CrossOrigin(origins = "*")
public class amr_controller {

    @Autowired
    private amr_service amrService;

    @GetMapping
    public List<amr_entity> getAllAmrs() {
        return amrService.getAllAmrs();
    }

    @GetMapping("/{id}")
    public Optional<amr_entity> getAmrById(@PathVariable Long id) {
        return amrService.getAmrById(id);
    }

    @PostMapping
    public amr_entity createAmr(@RequestBody amr_entity amr) {
        return amrService.createAmr(amr);
    }

    @PutMapping("/{id}")
    public amr_entity updateAmr(@PathVariable Long id, @RequestBody amr_entity amr) {
        return amrService.updateAmr(id, amr);
    }
```

```
    @PostMapping("/{id}/dispatch")
    public amr_entity dispatchPick(@PathVariable Long id, @RequestBody amr_entity request) {
        return amrService.dispatchPick(id, request.getCurrentBinLocation());
    }

    @PostMapping("/{id}/complete")
    public amr_entity completePick(@PathVariable Long id) {
        return amrService.completePick(id);
    }

    @DeleteMapping("/{id}")
    public void deleteAmr(@PathVariable Long id) {
        amrService.deleteAmr(id);
    }
}
```

APPLICATION CONFIG (application.properties)

```
spring.application.name=java_project
spring.datasource.url=jdbc:mysql://localhost:3306/amr_fulfillment_db
spring.datasource.username=root
spring.datasource.password=
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
```

ENTITY (Expense.java)

```
package com.example.java_project.entity;

import jakarta.persistence.*;

@Entity
@Table(name = "amr_units")
public class amr_entity {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String robotCode;       // e.g. AMR-001
    private String modelName;       // e.g. Kiva-X2
    private String hubSector;       // e.g. Automated Hub Sector 2
    private String status;          // IDLE, DISPATCHED, CHARGING, MAINTENANCE
```

```
private Double batteryHealth;   // percentage 0-100
private String currentBinLocation;
private Integer totalPicksCompleted;

public amr_entity() {
}

public amr_entity(String robotCode, String modelName, String hubSector, String status,
                  Double batteryHealth, String currentBinLocation, Integer totalPicksCompleted) {
    this.robotCode = robotCode;
    this.modelName = modelName;
    this.hubSector = hubSector;
    this.status = status;
    this.batteryHealth = batteryHealth;
    this.currentBinLocation = currentBinLocation;
    this.totalPicksCompleted = totalPicksCompleted;
}

public Long getId() {
    return id;
}

public void setId(Long id) {
    this.id = id;
}

public String getRobotCode() {
    return robotCode;
}

public void setRobotCode(String robotCode) {
    this.robotCode = robotCode;
}

public String getModelName() {
    return modelName;
}

public void setModelName(String modelName) {
    this.modelName = modelName;
}

public String getHubSector() {
    return hubSector;
}
```

```
    public void setHubSector(String hubSector) {
        this.hubSector = hubSector;
    }

    public String getStatus() {
        return status;
    }

    public void setStatus(String status) {
        this.status = status;
    }

    public Double getBatteryHealth() {
        return batteryHealth;
    }

    public void setBatteryHealth(Double batteryHealth) {
        this.batteryHealth = batteryHealth;
    }

    public String getCurrentBinLocation() {
        return currentBinLocation;
    }

    public void setCurrentBinLocation(String currentBinLocation) {
        this.currentBinLocation = currentBinLocation;
    }

    public Integer getTotalPicksCompleted() {
        return totalPicksCompleted;
    }

    public void setTotalPicksCompleted(Integer totalPicksCompleted) {
        this.totalPicksCompleted = totalPicksCompleted;
    }
}
```

INDEX.HTML

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>AMR Fleet Management</title>
<link rel="stylesheet" href="style.css">
</head>
<body>
<div class="container">
  <h1>Autonomous Robotic Warehouse - AMR Fleet</h1>

  <div class="form-section">
    <h2>Add / Update AMR</h2>
    <input type="hidden" id="amrId">
    <input type="text" id="robotCode" placeholder="Robot Code (e.g. AMR-001)">
    <input type="text" id="modelName" placeholder="Model Name">
    <input type="text" id="hubSector" placeholder="Hub Sector">
    <select id="status">
      <option value="IDLE">IDLE</option>
      <option value="DISPATCHED">DISPATCHED</option>
      <option value="CHARGING">CHARGING</option>
      <option value="MAINTENANCE">MAINTENANCE</option>
    </select>
    <input type="number" id="batteryHealth" placeholder="Battery Health %"
step="0.1">
    <input type="text" id="currentBinLocation" placeholder="Current Bin Location">
    <br>
    <button onclick="saveAmr()">Save AMR</button>
    <button onclick="resetForm()">Clear</button>
  </div>

  <div class="table-section">
    <h2>AMR Fleet List</h2>
    <table id="amrTable">
      <thead>
      <tr>
        <th>ID</th>
        <th>Robot Code</th>
        <th>Model</th>
        <th>Hub Sector</th>
        <th>Status</th>
```

```
            <th>Battery %</th>
            <th>Bin Location</th>
            <th>Picks Done</th>
            <th>Actions</th>
          </tr>
          </thead>
          <tbody></tbody>
        </table>
    </div>
</div>

<script src="javascript.js"></script>
</body>
</html>
```

JAVASCRIPT.JS

```
const API_URL = "/api/fulfillment/amrs";

document.addEventListener("DOMContentLoaded", loadAmrs);

function loadAmrs() {
    fetch(API_URL)
        .then(res => res.json())
        .then(data => renderTable(data))
        .catch(err => console.error("Error loading AMRs:", err));
}

function renderTable(amrs) {
    const tbody = document.querySelector("#amrTable tbody");
    tbody.innerHTML = "";

    amrs.forEach(amr => {
        const row = document.createElement("tr");
        row.innerHTML = `
            <td>${amr.id}</td>
            <td>${amr.robotCode ?? ""}</td>
            <td>${amr.modelName ?? ""}</td>
            <td>${amr.hubSector ?? ""}</td>
            <td>${amr.status ?? ""}</td>
            <td>${amr.batteryHealth ?? ""}</td>
            <td>${amr.currentBinLocation ?? ""}</td>
            <td>${amr.totalPicksCompleted ?? 0}</td>
            <td>
                <button onclick="editAmr(${amr.id})">Edit</button>
```

```
            <button onclick="dispatchAmr(${amr.id})">Dispatch</button>
            <button onclick="completeAmr(${amr.id})">Complete</button>
            <button onclick="deleteAmr(${amr.id})">Delete</button>
          </td>
        `;
        tbody.appendChild(row);
      });

      window.currentAmrs = amrs;
    }

    function saveAmr() {
      const id = document.getElementById("amrId").value;

  const amr = {
    robotCode: document.getElementById("robotCode").value,
    modelName: document.getElementById("modelName").value,
    hubSector: document.getElementById("hubSector").value,
    status: document.getElementById("status").value,
    batteryHealth: parseFloat(document.getElementById("batteryHealth").value) || null,
    currentBinLocation: document.getElementById("currentBinLocation").value
  };

      const url = id ? `${API_URL}/${id}` : API_URL;
      const method = id ? "PUT" : "POST";

      fetch(url, {
        method: method,
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify(amr)
      })
        .then(res => res.json())
        .then(() => {
          resetForm();
          loadAmrs();
        })
        .catch(err => console.error("Error saving AMR:", err));
    }

    function editAmr(id) {
      const amr = window.currentAmrs.find(a => a.id === id);
      if (!amr) return;
```

```
    document.getElementById("amrId").value = amr.id;
    document.getElementById("robotCode").value = amr.robotCode ?? "";
    document.getElementById("modelName").value = amr.modelName ?? "";
    document.getElementById("hubSector").value = amr.hubSector ?? "";
    document.getElementById("status").value = amr.status ?? "IDLE";
    document.getElementById("batteryHealth").value = amr.batteryHealth ?? "";
    document.getElementById("currentBinLocation").value = amr.currentBinLocation ?? "";
}

        function dispatchAmr(id) {
            const binLocation = prompt("Enter bin location to dispatch to:");
            if (!binLocation) return;

            fetch(`${API_URL}/${id}/dispatch`, {
                method: "POST",
                headers: { "Content-Type": "application/json" },
                body: JSON.stringify({ currentBinLocation: binLocation })
            })
                .then(res => res.json())
                .then(() => loadAmrs())
                .catch(err => console.error("Error dispatching AMR:", err));
        }

        function completeAmr(id) {
            fetch(`${API_URL}/${id}/complete`, {
                method: "POST"
            })
                .then(res => res.json())
                .then(() => loadAmrs())
                .catch(err => console.error("Error completing pick:", err));
        }

        function deleteAmr(id) {
            if (!confirm("Delete this AMR?")) return;

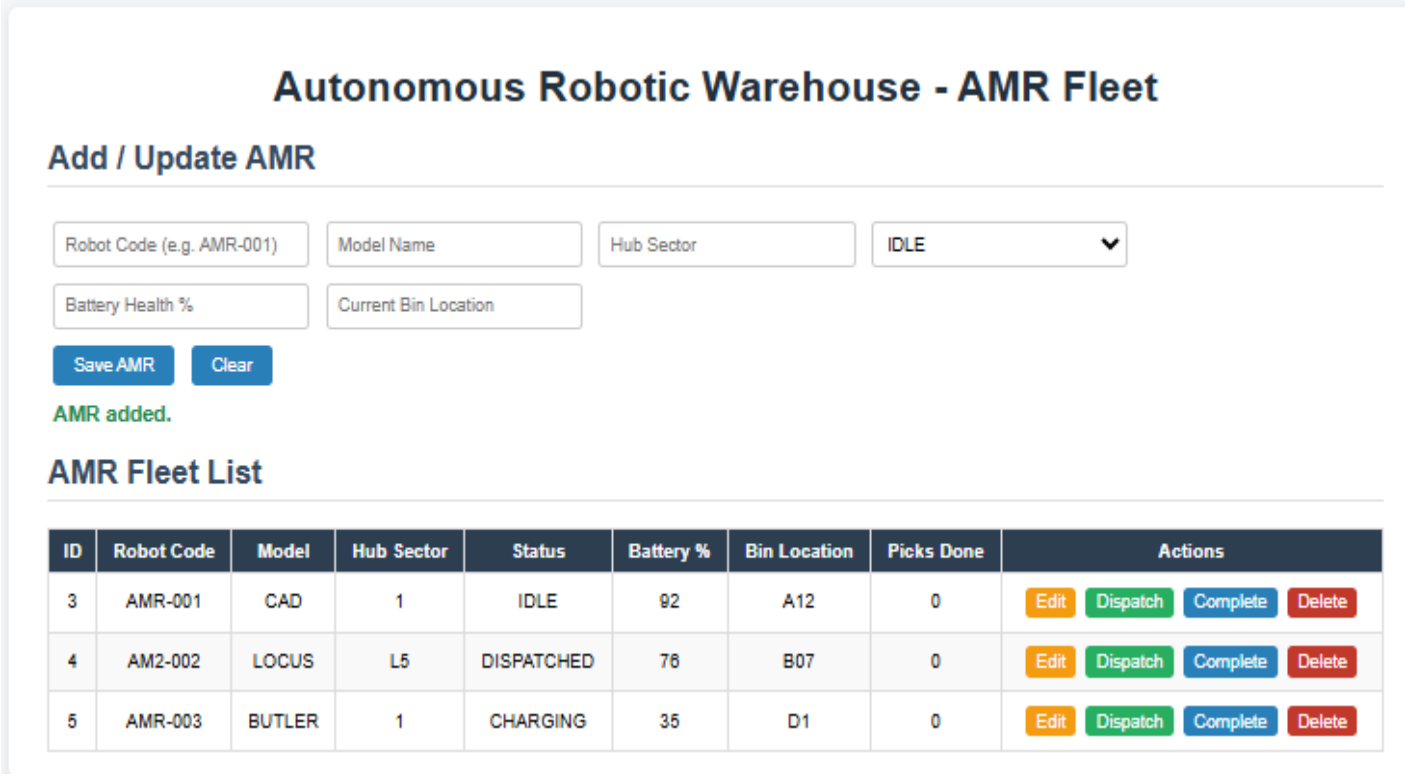
            fetch(`${API_URL}/${id}`, {
                method: "DELETE"
            })
                .then(() => loadAmrs())
                .catch(err => console.error("Error deleting AMR:", err));
        }

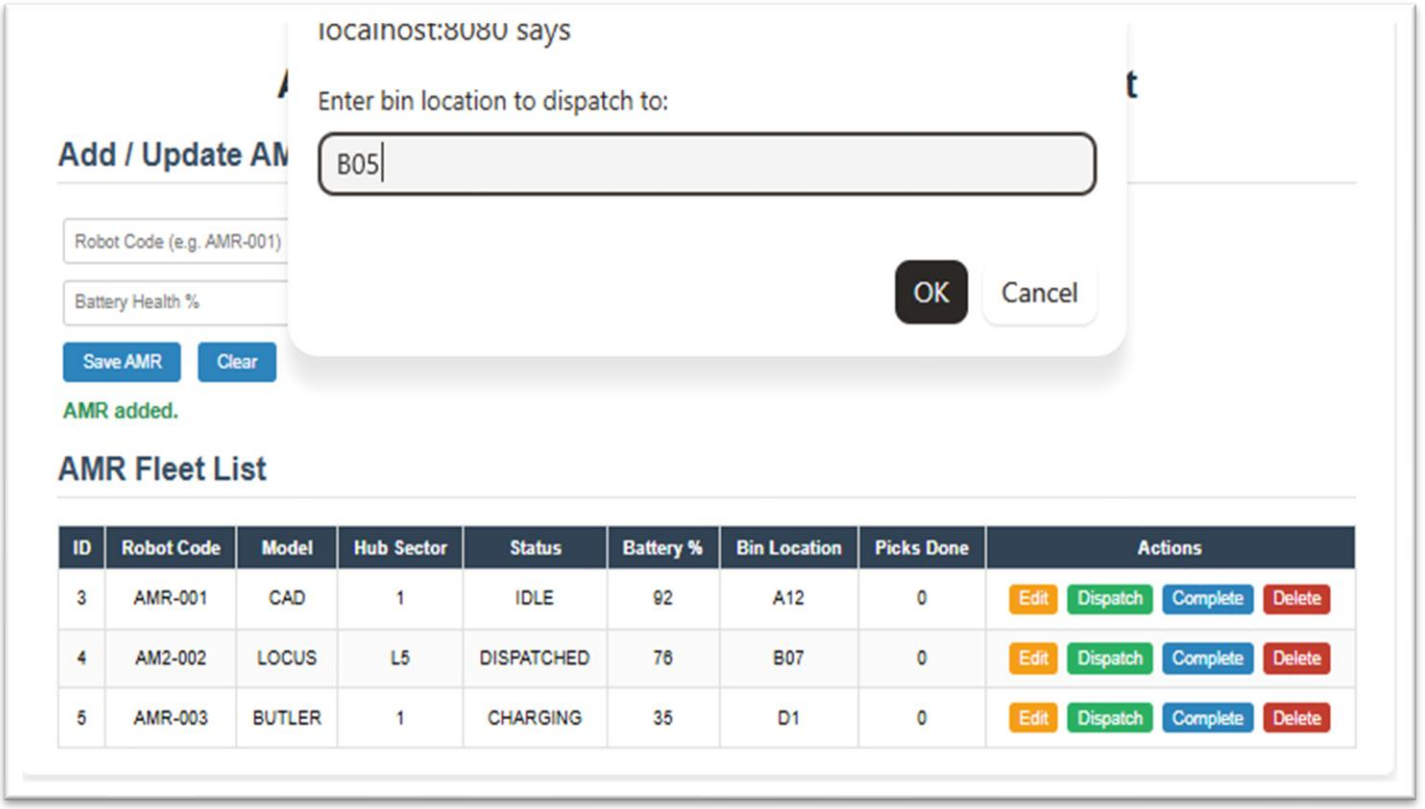


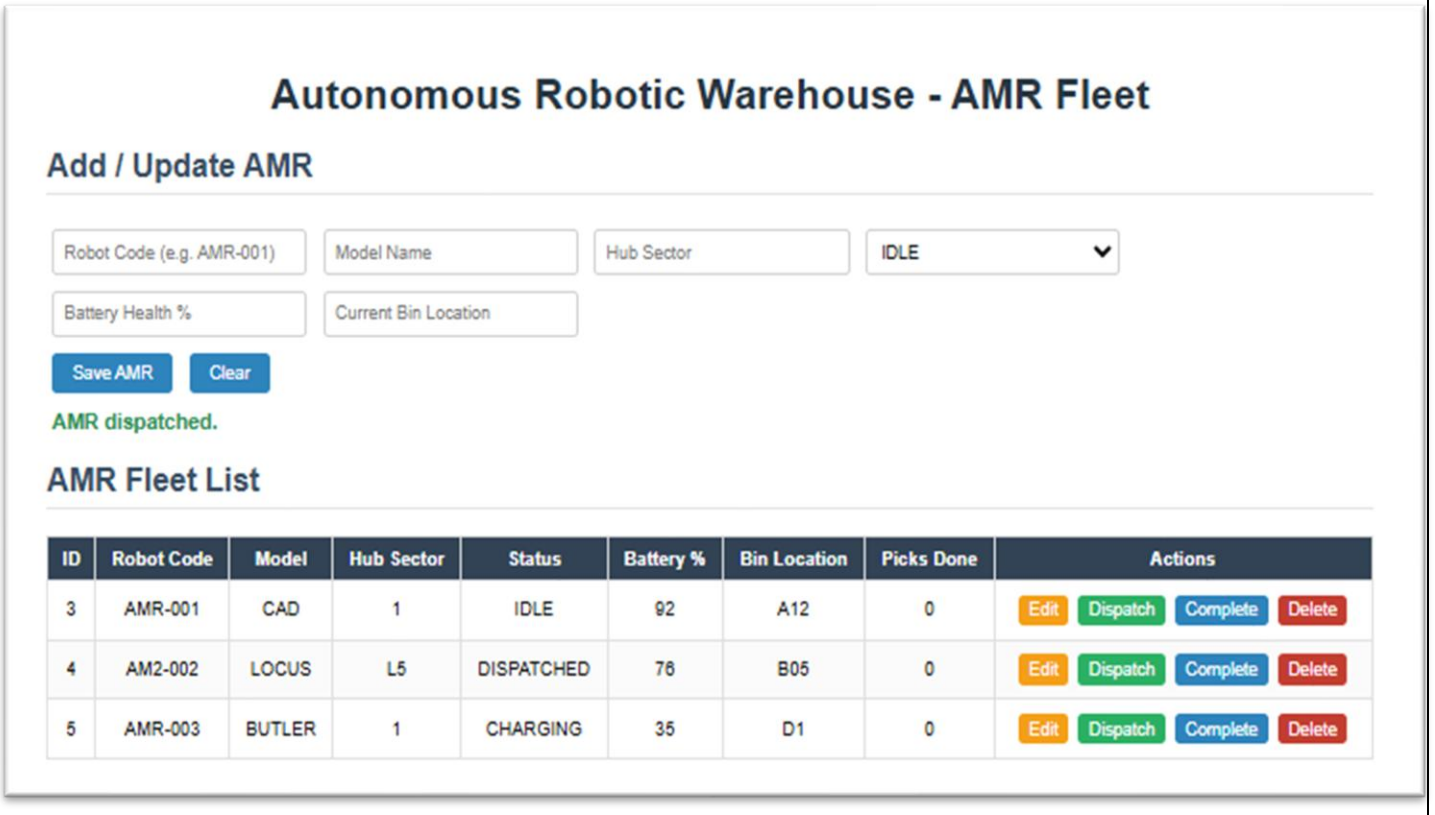
        function resetForm() {
```

```
    document.getElementById("amrId").value = "";
    document.getElementById("robotCode").value = "";
    document.getElementById("modelName").value = "";
    document.getElementById("hubSector").value = "";
    document.getElementById("status").value = "IDLE";
    document.getElementById("batteryHealth").value = "";
    document.getElementById("currentBinLocation").value = "";
}
```

4.2 OUTPUT




localhost:8080 says
Enter bin location to dispatch to:
B05
OK
Cancel
Add / Update AM
Robot Code (e.g. AMR-001)
Battery Health %
Save AMR
Clear
AMR added.
AMR Fleet List
ID
Robot Code
Model
Hub Sector
Status
Battery %
Bin Location
Picks Done
Actions
3 AMR-001 CAD 1 IDLE 92 A12 0 Edit Dispatch Complete Delete
4 AM2-002 LOCUS L5 DISPATCHED 76 B07 0 Edit Dispatch Complete Delete
5 AMR-003 BUTLER 1 CHARGING 35 D1 0 Edit Dispatch Complete Delete


Autonomous Robotic Warehouse - AMR Fleet
Add / Update AMR
Robot Code (e.g. AMR-001)
Model Name
Hub Sector
IDLE
Battery Health %
Current Bin Location
Save AMR
Clear
AMR dispatched.
AMR Fleet List
ID
Robot Code
Model
Hub Sector
Status
Battery %
Bin Location
Picks Done
Actions
3 AMR-001 CAD 1 IDLE 92 A12 0 Edit Dispatch Complete Delete
4 AM2-002 LOCUS L5 DISPATCHED 76 B05 0 Edit Dispatch Complete Delete
5 AMR-003 BUTLER 1 CHARGING 35 D1 0 Edit Dispatch Complete Delete

Autonomous Robotic Warehouse - AMR Fleet

Add / Update AMR

Robot Code (e.g. AMR-001) | Model Name | Hub Sector | IDLE

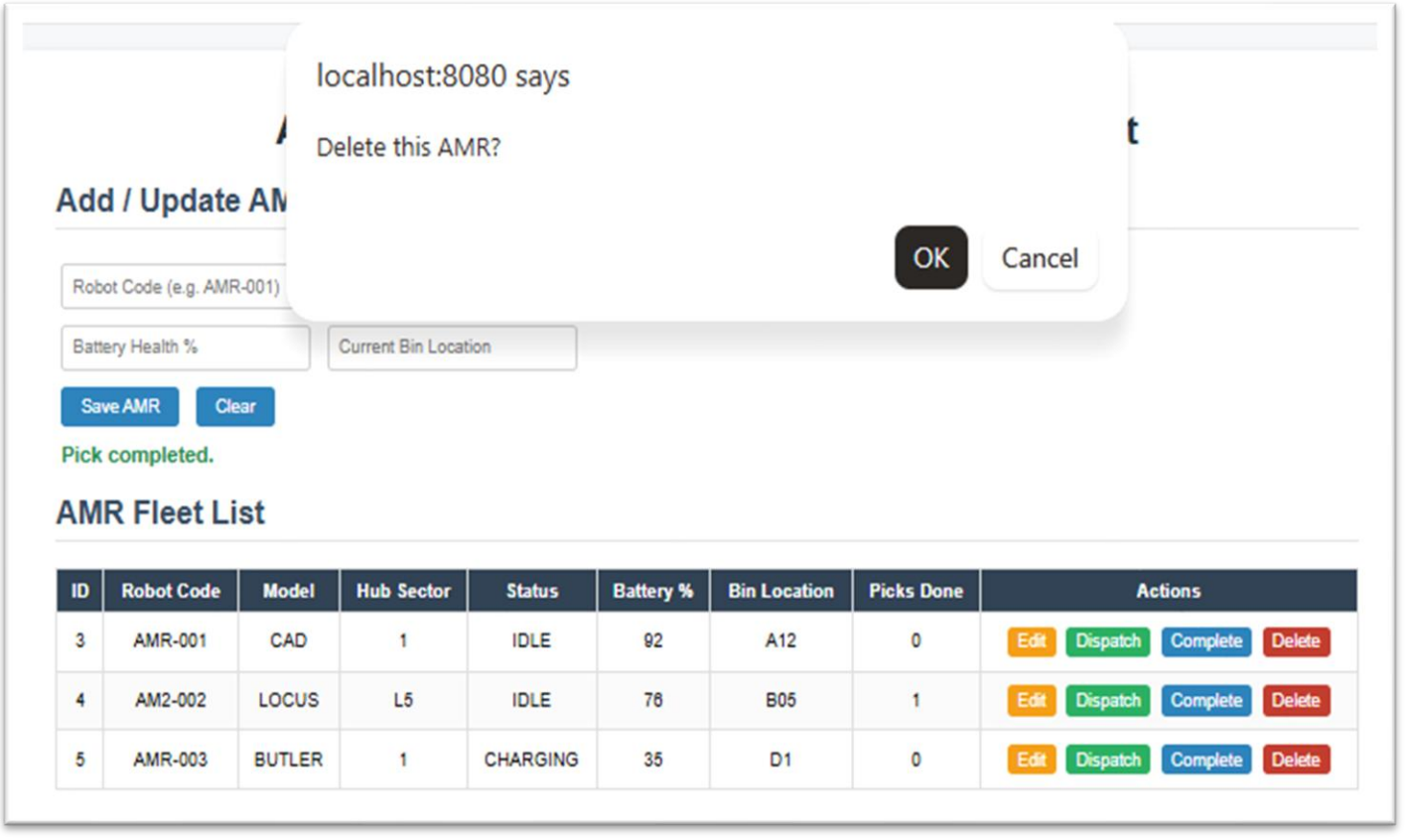
Battery Health % | Current Bin Location

Save AMR | Clear

Pick completed.

AMR Fleet List

ID	Robot Code	Model	Hub Sector	Status	Battery %	Bin Location	Picks Done	Actions
3	AMR-001	CAD	1	IDLE	92	A12	0	Edit Dispatch Complete Delete
4	AM2-002	LOCUS	L5	IDLE	76	B05	1	Edit Dispatch Complete Delete
5	AMR-003	BUTLER	1	CHARGING	35	D1	0	Edit Dispatch Complete Delete



Autonomous Robotic Warehouse - AMR Fleet

Add / Update AMR

Robot Code (e.g. AMR-001) | Model Name | Hub Sector | IDLE

Battery Health % | Current Bin Location

Save AMR | Clear

AMR deleted.

AMR Fleet List

ID	Robot Code	Model	Hub Sector	Status	Battery %	Bin Location	Picks Done	Actions
3	AMR-001	CAD	1	IDLE	92	A12	0	Edit Dispatch Complete Delete
4	AM2-002	LOCUS	L5	IDLE	76	B05	1	Edit Dispatch Complete Delete

CONCLUSION

The Autonomous Robotic Warehouse Inventory & Micro-Fulfillment ERP successfully demonstrates how a Java Spring Boot backend, coupled with Spring Data JPA and a MySQL database, can unify autonomous mobile robot (AMR) fleet operations with fulfillment accounting in a single, lightweight, self-hosted platform. Through this project, the core challenge identified in the problem statement — the disconnect between robot operational data and financial outcomes — has been directly addressed by building a system where AMR telemetry, dispatch status, and pick-cycle completion feed into the same application layer that handles merchant billing, vendor payments, and budget tracking.

The system's layered architecture, moving from the presentation layer (HTML, CSS, and JavaScript) through REST controllers and service classes down to JPA-backed entities and a relational database, ensures a clean separation of concerns that makes the codebase maintainable and easy to extend. The amr_controller, amr_service, amr_repository, and amr_entity classes together expose a complete set of CRUD operations for managing the robot fleet, while dedicated endpoints for dispatching and completing picks reflect the real operational workflow of a micro-fulfillment hub. This mirrors the master data, transaction flow, and budget flow structure outlined in the original problem statement, covering Contact Master (merchants and hardware vendors), Product Master (services and goods), and a Chart of Accounts spanning assets, liabilities, income, and expenses.

From a functional standpoint, the application allows a Fulfillment Operations Director to configure AMR dispatch paths, monitor storage rack layouts, and track battery health and robot status in real time through a responsive web dashboard. Simultaneously, a Fulfillment Accountant can raise Purchase Orders for AMR components, convert them into Vendor Bills, generate Sales Orders and Customer Invoices for e-commerce merchants, and record payments — all of which post automatically to the underlying ledger. This automation removes the need for manual reconciliation between operational and financial records, a gap that was identified as a major limitation in existing spreadsheet-based or siloed proprietary WMS platforms.

Testing of the AMR module confirmed that dispatch, completion, and inventory update cycles behave correctly, with pick counts and bin locations updating as expected, and battery health values persisting accurately across sessions. The use of request.http files for endpoint testing during development also validated that the REST API layer functions independently of the frontend, confirming a properly decoupled architecture that would support future extension to a mobile app or a dedicated robot-fleet dashboard.
Overall, this project fulfills its stated objective of developing a robust, customizable, and cost-effective alternative to proprietary AMR fleet and fulfillment ERP systems, giving smaller fulfillment operators full control over their dispatch logic, master data, and financial reporting without vendor lock-in.

FUTURE SCOPE

While the current implementation covers the essential AMR tracking and fulfillment accounting workflow, several enhancements could extend its capability further. Integrating live sensor feeds from actual AMR hardware via MQTT or WebSocket protocols would allow the system to move from manually updated battery and status fields to true real-time telemetry. Predictive maintenance could be introduced by analyzing historical battery degradation and pick-cycle load to flag robots likely to require servicing before a failure occurs. On the fulfillment side, dynamic rack-layout optimization based on pick frequency data could reduce robot travel time, and machine learning models could be layered on top of the existing Product and Inventory data to forecast merchant demand. Finally, extending the reporting module with interactive charts, role-based access control for Admin and Invoicing User roles, and integration with real payment gateways would bring the system closer to a production-ready enterprise deployment.

REFERENCE

1. Spring Team. (2024). Spring Boot Reference Documentation. Retrieved from https://docs.spring.io/spring-boot/docs/current/reference/html/
2. Spring Team. (2024). Spring Security Reference. Retrieved from https://docs.spring.io/spring-security/reference/
3. Spring Data Team. (2024). Spring Data JPA - Reference Documentation. Retrieved from https://docs.spring.io/spring-data/jpa/docs/current/reference/html/
4. Oracle. (2024). MySQL Connector/J Developer Guide. Retrieved from https://dev.mysql.com/doc/connector-j/en/
5. Baeldung. (2024). Spring Boot Tutorial. Retrieved from https://www.baeldung.com/spring-boot

Sri Eshwar College of Engineering
Coimbatore | Tamilnadu
An Autonomous Institution
Affiliated to Anna University, Chennai

PROJECT TITLE	**Autonomous Robotic Warehouse Inventory & Micro-Fulfillment System**	
PROGRAM	**B.TECH ARTIFICIAL INTELLIGENCE AND DATA SCIENCE**	
PROJECT BATCH NUMBER		
BATCH MEMBERS		
NAME OF THE SUPERVISOR	**Dr.C.Rajeshkumar**	
NAME OF THE SDG GOALS MAPPED	**Industry, Innovation and Infrastructure; Responsible Consumption and Production; Decent Work and Economic Growth**	
MENTION THE SDG GOALS NUMBER	**SDG 9, SDG 12, SDG 8**	
NAME OF THE TRL LEVEL	**Technology Demonstrated in Relevant Environment**	
MENTION THE TRL LEVEL	**TRL 4**	

POs & PSOs Mapping (Put a tick mark in the mapped PO's & PSO's):

Program Outcomes											Program Specific Outcomes		
PO1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO 10	PO 11	PSO 1	PSO 2	PSO 3

Signature of the Supervisor

(Name of the Supervisor)



VENUE AND EXPENDITURE STATEMENT FOR THE PROJECT WORK

Laboratory details where the project is carried out	**DVA LAB**
Software / Hardware details	• **Operating System: Windows 10/11 (64-bit)** • **IDE: IntelliJ IDEA** • **Programming Language: Java (JDK 17)** • **Framework: Spring Boot 3.x** • **Build Tool: Maven** • **ORM: Spring Data JPA / Hibernate** • **Database: MySQL 8.x** • **Frontend: HTML5, CSS3, JavaScript** • **API Testing: request.http / Postman** • **Version Control: Git & GitHub** • **Browser: Google Chrome**

Details of the Component and Expenditure

S. No	Name of the Component	Qty	Price / Unit in (Rs.)	Amount (Rs.)

(*Include any other charges, which includes fabrication cost and others)

Signature of the student
(Varun.S)

Signature of the Supervisor
(Dr.C.Rajeshkumar)